

岗位能力图谱数据库 Schema

1. 文档说明

本文档定义“生涯棱镜”岗位能力图谱第一版数据库 Schema，作为数据清洗、实体标准化、图谱写入、查询接口和可视化开发的统一依据。

第一版遵循最小可用原则，仅保存岗位分类、岗位能力和技能层级关系。原始招聘信息、完整证据文本、简历、用户信息和人工审核记录保存在业务数据库或数据文件中，不作为图谱节点。

2. 总体结构

(Position)–[IN_CATEGORY]–>(PositionCategory)
(Position)–[REQUIRES]–>(Skill)
(Skill)–[BELONGS_T0]–>(SkillCluster)
(SkillCluster)–[BELONGS_T0]–>(TechStack)

Schema 共包含：

- 5 类节点：Position、PositionCategory、Skill、SkillCluster、TechStack
- 4 种关系模式：IN_CATEGORY、REQUIRES、两级 BELONGS_T0

第一版不建立岗位之间或技能之间的直接关系。

3. 节点定义

3.1 Position

表示经过名称归一化后的标准岗位，不表示某家公司发布的具体招聘记录。

属性	类型	必填	唯一	说明
position_id	String	是	是	岗位唯一标识，如 pos_java_backend
name	String	是	是	标准岗位名称
aliases	List<String>	否	否	岗位别名和常见写法
description	String	否	否	岗位定义
status	String	是	否	岗位状态
first_seen	Date	是	否	首次在数据中出现的日期
last_seen	Date	是	否	最近在数据中出现的日期
created_at	DateTime	是	否	节点创建时间
updated_at	DateTime	是	否	节点更新时间
JD	String	是	否	数据源，原始内容

status 取值:

值	含义
existing	既有岗位
emerging	新兴岗位
inactive	当前样本中不再活跃

示例:

```
{
  "position_id": "pos_ai_agent_engineer",
  "name": "AI Agent研发工程师",
  "aliases": ["智能体研发工程师", "Agent开发工程师"],
  "description": "负责智能体应用、工具调用及 workflows 系统研发",
  "status": "emerging",
  "first_seen": "2025-03-01",
  "last_seen": "2026-07-29"
}
```

3.2 PositionCategory

表示职责和技能结构相近的一组岗位。岗位类别由岗位技能集合聚类得到，并经过人工命名或确认，不直接采用招聘平台原始分类。

属性	类型	必填	唯一	说明
category_id	String	是	是	类别唯一标识
name	String	是	是	类别名称
description	String	否	否	类别定义
keywords	List<String>	否	否	类别代表性关键词
created_at	DateTime	是	否	节点创建时间
updated_at	DateTime	是	否	节点更新时间

示例：

```
{
  "category_id": "category_ai_engineering",
  "name": "人工智能研发",
  "description": "从事机器学习、大模型、算法系统及智能应用研发的岗位类别",
  "keywords": ["机器学习", "深度学习", "大模型", "算法", "智能体"]
}
```

建议首批岗位类别：

- 后端研发
- 前端研发
- 客户端研发
- 人工智能研发
- 数据研发
- 测试与质量保障
- 云计算与运维
- 网络与信息安全
- 数据库研发
- 物联网与嵌入式研发
- 产品与项目管理
- 交互与视觉设计

3.3 Skill

表示能够从 JD 或简历中识别并用于匹配的最小技能单元。

属性	类型	必填	唯一	说明
skill_id	String	是	是	技能唯一标识，如 skill_python
name	String	是	是	标准技能名称
aliases	List<String>	否	否	缩写、别名及不同写法
description	String	否	否	技能定义
skill_type	String	是	否	技能类型
status	String	是	否	技能状态
created_at	DateTime	是	否	节点创建时间
updated_at	DateTime	是	否	节点更新时间

skill_type 取值：

值	含义
language	编程语言
framework	框架或开发库
tool	工具或平台
database	数据库
method	方法、算法或工程能力
knowledge	理论或领域知识
soft_skill	通用能力

status 取值: active、inactive、pending。

示例：

```
{
  "skill_id": "skill_langchain",
  "name": "LangChain",
  "aliases": ["langchain"],
  "description": "用于构建大语言模型应用的开发框架",
  "skill_type": "framework",
  "status": "active"
}
```

技能粒度应满足可识别、可匹配和含义明确三个条件。例如，Python、LangChain、机器学习可以作为技能点；“人工智能技术”“熟悉相关技术”等宽泛表述不作为技能点。

3.4 SkillCluster

表示用途或能力方向相近的一组技能。

属性	类型	必填	唯一	说明
cluster_id	String	是	是	技能簇唯一标识
name	String	是	是	技能簇名称
description	String	否	否	技能簇定义
created_at	DateTime	是	否	节点创建时间
updated_at	DateTime	是	否	节点更新时间

示例：

```
{
  "cluster_id": "cluster_llm_application",
  "name": "大模型应用开发",
  "description": "围绕大模型调用、智能体、知识库和工作流开发的技能集合"
}
```

3.5 TechStack

表示图谱最上层的技术领域，用于全景分类和视图切换。

属性	类型	必填	唯一	说明
stack_id	String	是	是	技术栈唯一标识
name	String	是	是	技术栈名称
description	String	否	否	技术栈定义
created_at	DateTime	是	否	节点创建时间
updated_at	DateTime	是	否	节点更新时间

建议首批技术栈：

软件开发
人工智能
大数据
云计算与运维
网络与信息安全
物联网与嵌入式
数据库与数据管理
产品与设计
通用职业能力

4. 关系定义

4.1 IN_CATEGORY

(Position)-[IN_CATEGORY]->(PositionCategory)
--

表示标准岗位所属的岗位类别。

第一版约束：

- 每个 Position 必须归属于一个 PositionCategory
- 每个 Position 只能归属于一个 PositionCategory
- 关系不设置业务属性

示例：

(AI Agent研发工程师)-[:IN_CATEGORY]->(人工智能研发)
--

4.2 REQUIRES

(Position)-[REQUIRES]->(Skill)

表示标准岗位对某项技能的要求，是图谱中的核心业务关系。

属性	类型	必填	说明
requirement_type	String	是	要求类型
weight	Float	是	技能对岗位的重要程度，范围为 0~1
frequency	Float	是	技能在该岗位有效 JD 中的出现比例
confidence	Float	是	岗位与技能关系的可信度
sample_count	Integer	是	支撑该关系的有效 JD 数量
first_seen	Date	是	首次识别到该关系的日期
last_seen	Date	是	最近识别到该关系的日期
trend	String	是	当前变化趋势
source_ids	List<String>	是	对应的原始 JD 编号
updated_at	DateTime	是	关系更新时间

requirement_type 取值:

值	含义
required	必备技能
preferred	加分技能

trend 取值:

值	含义
new	新增能力
rising	需求上升
stable	基本稳定
declining	需求下降

数值与时间约束:

```
0 <= weight <= 1
0 <= frequency <= 1
0 <= confidence <= 1
sample_count >= 1
first_seen <= last_seen
```

每一组 Position 和 Skill 之间最多存在一条 REQUIRES 关系。

示例：

```
{
  "requirement_type": "required",
  "weight": 0.86,
  "frequency": 0.72,
  "confidence": 0.91,
  "sample_count": 43,
  "first_seen": "2025-06-10",
  "last_seen": "2026-07-29",
  "trend": "rising",
  "source_ids": ["jd_1021", "jd_1035", "jd_1088"],
  "updated_at": "2026-07-29T15:30:00+08:00"
}
```

4.3 BELONGS_TO：技能归属技能簇

```
(Skill)-[BELONGS_TO]->(SkillCluster)
```

第一版约束：

- 每个 Skill 必须归属于一个 SkillCluster
- 每个 Skill 只能归属于一个 SkillCluster
- 关系不设置业务属性

示例：

```
(LangChain)-[:BELONGS_TO]->(大模型应用开发)
```

4.4 BELONGS_TO：技能簇归属技术栈

```
(SkillCluster)-[BELONGS_TO]->(TechStack)
```

第一版约束：

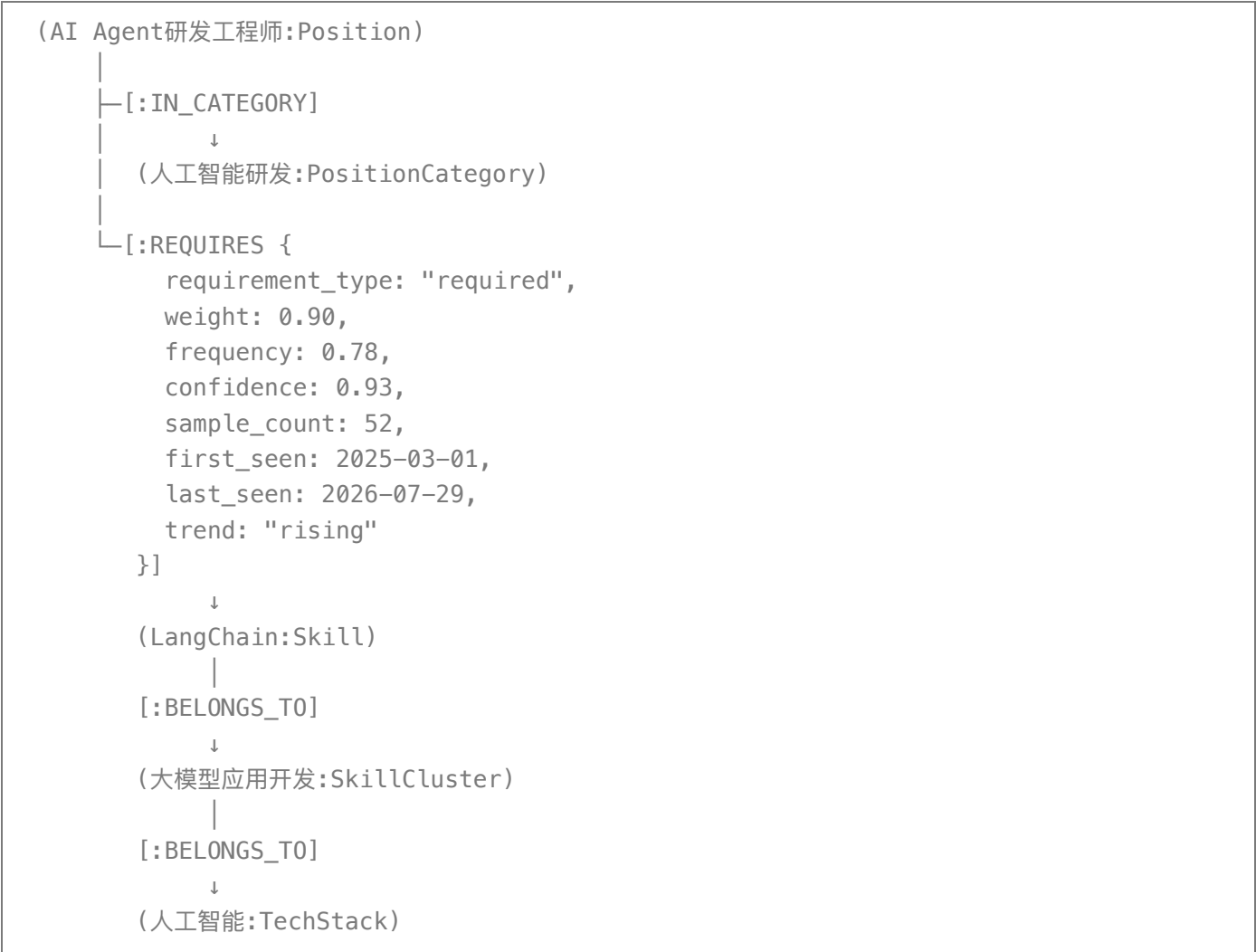
- 每个 SkillCluster 必须归属于一个 TechStack
- 每个 SkillCluster 只能归属于一个 TechStack
- 关系不设置业务属性

示例：

(大模型应用开发)-[:BELONGS_TO]->(人工智能)

两级关系使用相同的 BELONGS_TO 名称，但起止节点类型不同，不产生语义冲突。

5. 完整实例



6. 标识符规范

所有内部标识符使用小写英文、数字和下划线，不使用名称本身作为主键。

Position	pos_<英文语义名称>
PositionCategory	category_<英文语义名称>
Skill	skill_<英文语义名称>
SkillCluster	cluster_<英文语义名称>
TechStack	stack_<英文语义名称>

示例：

```
pos_java_backend
category_backend_engineering
skill_spring_boot
cluster_java_ecosystem
stack_software_development
```

节点名称变化时保持内部标识符不变。

7. Neo4j 约束与索引

```
CREATE CONSTRAINT position_id_unique IF NOT EXISTS
FOR (n:Position) REQUIRE n.position_id IS UNIQUE;

CREATE CONSTRAINT position_category_id_unique IF NOT EXISTS
FOR (n:PositionCategory) REQUIRE n.category_id IS UNIQUE;

CREATE CONSTRAINT skill_id_unique IF NOT EXISTS
FOR (n:Skill) REQUIRE n.skill_id IS UNIQUE;

CREATE CONSTRAINT skill_cluster_id_unique IF NOT EXISTS
FOR (n:SkillCluster) REQUIRE n.cluster_id IS UNIQUE;

CREATE CONSTRAINT tech_stack_id_unique IF NOT EXISTS
FOR (n:TechStack) REQUIRE n.stack_id IS UNIQUE;

CREATE INDEX position_name_index IF NOT EXISTS
FOR (n:Position) ON (n.name);

CREATE INDEX position_category_name_index IF NOT EXISTS
FOR (n:PositionCategory) ON (n.name);

CREATE INDEX skill_name_index IF NOT EXISTS
FOR (n:Skill) ON (n.name);

CREATE INDEX skill_cluster_name_index IF NOT EXISTS
FOR (n:SkillCluster) ON (n.name);

CREATE INDEX tech_stack_name_index IF NOT EXISTS
FOR (n:TechStack) ON (n.name);
```

Neo4j 不能仅通过上述约束保证“每个节点只能有一条归属关系”，该规则由图谱写入模块负责校验。

8. 图谱外数据

以下内容不进入第一版图谱：

- 公司、招聘平台和招聘人员
- 数据清洗与实体抽取中间结果

- 完整证据文本和人工审核记录
- 用户、简历和匹配结果
- 学习资源和学习路径

图谱中的 `REQUIRES.source_ids` 保存原始 JD 的业务主键。需要查看来源时，通过该字段查询图谱外的原始数据。

9. 新岗位定义与既有岗位更新规则

每次导入新一批 JD 时，系统先完成岗位名称标准化、技能抽取和技能标准化，再根据标准岗位库执行“新岗位判断”或“既有岗位更新”。两类流程的结果都写入同一套 Schema，不增加新节点类型。

9.1 通用预处理

1. 对 JD 标题进行清洗，去除地点、级别、部门、编号和招聘宣传词。
2. 将岗位别名映射为候选标准岗位名称。
3. 从岗位职责和岗位要求中抽取技能，并映射到标准 Skill。
4. 合并重复 JD，保留来源编号和有效发布时间。
5. 按时间窗口聚合同类 JD，计算岗位的技能集合、技能频次和样本数。

9.2 新岗位发现与定义

当候选岗位无法可靠映射到已有 Position 时，进入新岗位判断流程。

判断依据：

指标	说明
岗位名称新颖度	标准化名称是否与现有岗位名称及别名明显不同
技能组合差异度	技能集合是否与最相近既有岗位存在显著差异
持续出现情况	是否在连续时间窗口中重复出现，而非一次性噪声
样本支持度	是否有足够数量的有效 JD 支撑
来源一致性	是否由多个企业或多个数据来源共同支持
信息完整度	是否能够提取明确的职责、必备技能和加分技能

满足新岗位判定条件后，系统生成候选岗位定义：

标准岗位名称
岗位别名
岗位描述
所属岗位类别
必备技能
加分技能
首次出现时间
最近出现时间
支撑样本数
原始数据来源
判定置信度

候选岗位经人工确认后写入图谱：

1. 创建新的 `Position`，设置 `status = emerging`。
2. 使用样本中的最早发布时间设置 `first_seen`。
3. 使用最近发布时间设置 `last_seen`。
4. 根据岗位技能结构聚类并创建 `IN_CATEGORY`；岗位类别不存在时先提交人工确认，再创建新的 `PositionCategory`。
5. 为必备技能和加分技能分别创建 `REQUIRES`。
6. 在 `REQUIRES` 中保存 `requirement_type`、`weight`、`frequency`、`confidence`、`sample_count` 和 `source_ids`。
7. 新岗位的初始技能关系设置 `trend = new`。

未达到判定条件或未通过人工确认的候选岗位不写入正式图谱，保留在业务数据库的待审核记录中。

当新岗位经过预设观察周期、样本量达到稳定要求并再次通过审核后，将 `Position.status` 从 `emerging` 更新为 `existing`。

9.3 既有岗位能力更新

当标准化岗位能够映射到已有 `Position` 时，进入既有岗位更新流程。

系统以相同长度的当前时间窗口和历史时间窗口为基础，比较各技能的出现频率、重要度和样本支持度，形成新增、修改、下降和失活四类更新。

新增能力

当前时间窗口中出现新的标准技能，且达到最低频次、样本数和置信度要求：

1. 创建新的 `REQUIRES` 关系。
2. 设置 `first_seen` 和 `last_seen`。
3. 设置 `trend = new`。
4. 保存支撑该能力的 `source_ids`。

能力修改

已有技能的要求类型、重要度或出现频率发生有效变化：

1. 更新 `requirement_type`、`weight`、`frequency` 和 `confidence`。
2. 更新 `sample_count`、`source_ids`、`last_seen` 和 `updated_at`。
3. 需求显著上升时设置 `trend = rising`。
4. 变化不显著时设置 `trend = stable`。
5. 需求显著下降时设置 `trend = declining`。

能力下降

技能仍在当前窗口出现，但频率或重要度持续下降：

1. 保留 `REQUIRES` 关系。
2. 更新当前统计值和最近证据。
3. 设置 `trend = declining`。

能力失活

技能在连续多个时间窗口中均未达到最低支持要求：

1. 不立即删除 `REQUIRES` 关系。
2. 将 `frequency` 和 `weight` 更新为当前统计结果。
3. 设置 `trend = declining`。
4. 保留历史 `first_seen`、最后有效的 `last_seen` 和来源信息。
5. 查询当前有效岗位画像时过滤失活关系；历史演化查询仍可使用该关系。

第一版不创建独立的能力变更节点。新增、修改和下降状态由 `REQUIRES` 当前属性与历史快照比较得到，完整变更明细保存在业务数据库中。

9.4 岗位自身更新

每次处理既有岗位时：

1. 合并新发现且经确认的岗位别名到 `Position.aliases`。
2. 根据最新有效 JD 更新 `Position.description`，但保留历史版本于业务数据库。
3. 更新 `Position.last_seen` 和 `updated_at`。
4. 重新计算岗位类别；只有聚类结果稳定且人工确认后，才修改 `IN_CATEGORY`。
5. 岗位在连续多个时间窗口中没有有效样本时，将 `status` 设置为 `inactive`，但不删除节点及其关系。
6. 已失活岗位重新出现并通过有效性检查时，将 `status` 恢复为 `existing`。

9.5 阈值配置

新岗位判定、技能新增、趋势变化和岗位失活所需的时间窗口、最低频次、最低样本数、最低来源数和置信度阈值均由评测实验确定，并保存在图谱外的系统配置中，不写死在 `Schema` 中。

10. 版本结论

岗位能力图谱 Schema V1 最终确定为:

节点:

Position
PositionCategory
Skill
SkillCluster
TechStack

关系:

Position -[IN_CATEGORY]-> PositionCategory
Position -[REQUIRES]-> Skill
Skill -[BELONGS_TO]-> SkillCluster
SkillCluster -[BELONGS_TO]-> TechStack